Geometric Parameters

MATLAB Session 2 — Lecture Notes

Introduction

To define the problem completely, several physical and numerical parameters must be specified, including the domain geometry, boundary conditions, material properties, and discretization parameters.

1. Geometry
2. Node coordinates
3. Cell centroids
4. Cell volumes
5. Surface vectors
6. Face centers
7. Center-to-center vectors (**CF**)
8. Interpolation (weighting) factors
9. Interpolation of field values to faces
10. Cell gradients

Good Practices

- At the top of every function, document what it does, its inputs, and its outputs. You can also add some notes, like assumptions and limitations.
- If external library functions are used (i.e. code that is not your original work), add a reference and note if it was modified.
- If an AI tool was used in any function or script, add a note stating so and include the exact prompt that was used.

Notation and Mesh Conventions

Consider a structured quadrilateral mesh. If the number of *nodes* in the x- and y-directions is n_x and n_y, respectively, then the number of *elements* (cells) is

$$N_x = n_x - 1, \qquad N_y = n_y - 1$$

Node coordinates are stored in arrays $X(i, j)$ and $Y(i, j)$ with $i = 1, \ldots, n_x$ and $j = 1, \ldots, n_y$. The index i (rows) runs in the x-direction and j (columns) in the y-direction.

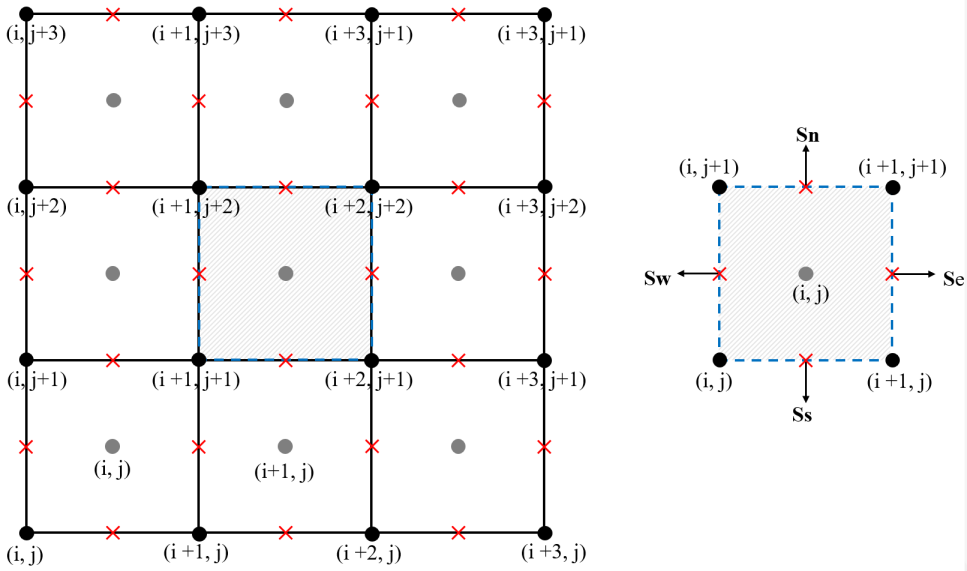

Figure 1: Domain

Centroids

The centroid of a quadrilateral cell (i, j) can be simply approximated as the average of its four corner nodes:

$$X_C(i,j) = \frac{1}{4}\Big[X(i,j) + X(i{+}1,j) + X(i,j{+}1) + X(i{+}1,j{+}1)\Big],$$
$$Y_C(i,j) = \frac{1}{4}\Big[Y(i,j) + Y(i{+}1,j) + Y(i,j{+}1) + Y(i{+}1,j{+}1)\Big] \tag{1}$$

```
function [XC, YC] = getCentroids(X, Y, Nx, Ny)
    % Initialize arrays
    XC = zeros(Nx, Ny);
    YC = zeros(Nx, Ny);
    % Loop over elements
    for i = 1:Nx
        for j = 1:Ny
            XC(i,j) = (X(i,j) + X(i+1,j) + X(i,j+1) + X(i+1,j+1)) / 4;
            YC(i,j) = (Y(i,j) + Y(i+1,j) + Y(i,j+1) + Y(i+1,j+1)) / 4;
        end
    end
end
```

Listing 1: Function to compute coordinates of cell centers (centroids)

Cell Volume

Each quadrilateral control volume is divided into two triangles by introducing a diagonal. The total area of the quadrilateral is then obtained as the sum of the areas of the two triangles. The area of a triangle formed by two vectors **a** and **b** can be computed using the cross product. Geometrically, the magnitude of the cross product $|\mathbf{a}\times\mathbf{b}|$ represents the area of the parallelogram spanned by the two vectors. Since a triangle occupies exactly half of this parallelogram, its area is given by

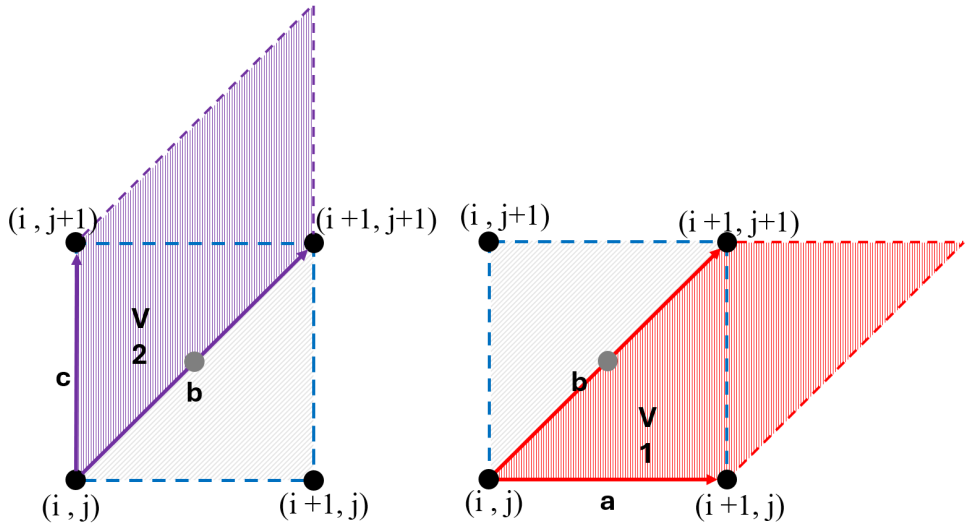

Figure 2: Domain

$$A_{\triangle} = \frac{1}{2}|\mathbf{a} \times \mathbf{b}|. \tag{2}$$

For cell (i, j), label the four vertices as

$$(x_1, y_1) = (X(i, j),\ Y(i, j)), \quad (x_2, y_2) = (X(i{+}1, j),\ Y(i{+}1, j)),$$

$$(x_3, y_3) = (X(i{+}1, j{+}1),\ Y(i{+}1, j{+}1)), \quad (x_4, y_4) = (X(i, j{+}1),\ Y(i, j{+}1)).$$

The quadrilateral is divided along the diagonal connecting nodes (x_1, y_1) and (x_3, y_3). The area of the first triangle, formed by nodes $(1, 2, 3)$, is

$$A_1 = \frac{1}{2}\,|(x_2 - x_1)(y_3 - y_1) - (y_2 - y_1)(x_3 - x_1)| \tag{3}$$

The area of the second triangle, formed by nodes $(1, 3, 4)$, is

$$A_2 = \frac{1}{2}\,|(x_3 - x_1)(y_4 - y_1) - (y_3 - y_1)(x_4 - x_1)| \tag{4}$$

The total area is therefore

$$A_{\text{cell}}(i, j) = A_1 + A_2 \tag{5}$$

```
function [elemVolume] = getVolume(X, Y, Nx, Ny)
    % Initialize arrays
    elemVolume = zeros(Nx, Ny);
    % Loop over elements
    for i = 1:Nx
        for j = 1:Ny
            x1 = X(i,j);      y1 = Y(i,j);
            x2 = X(i+1,j);    y2 = Y(i+1,j);
            x3 = X(i+1,j+1);  y3 = Y(i+1,j+1);
            x4 = X(i,j+1);    y4 = Y(i,j+1);
            % Assuming unit depth
            A1 = 0.5*abs((x2-x1)*(y3-y1) - (y2-y1)*(x3-x1));
            A2 = 0.5*abs((x3-x1)*(y4-y1) - (y3-y1)*(x4-x1));
```

```
            elemVolume(i,j) = (A1 + A2) * 1 ; % assuming unit depth
        end
    end
end
```

Listing 2: Function to compute element volumes

Surface Vectors

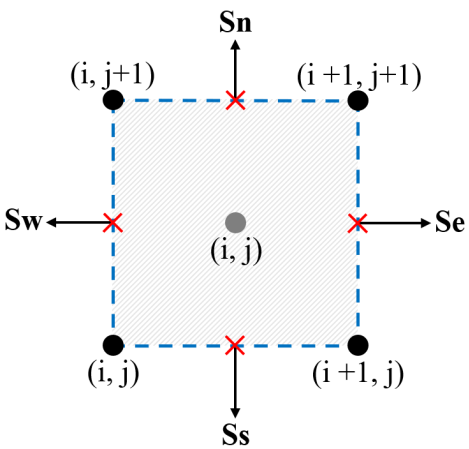

Figure 3: Domain

The surface vector is a vector perpendicular to the face (i.e., perpendicular to the tangent of the face), with magnitude equal to the face length and direction outward normal to the control volume.

In two dimensions, the face is represented by an edge. Consider the **east face** with endpoints $A(x_1, y_1) = (X(i+1, j),\ Y(i+1, j))$, $B(x_2, y_2) = (X(i+1, j+1),\ Y(i+1, j+1))$. The tangent vector along the face can be defined as:

$$\mathbf{AB} = \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \end{bmatrix} \qquad \text{or} \qquad \mathbf{BA} = \begin{bmatrix} x_1 - x_2 \\ y_1 - y_2 \end{bmatrix} \tag{6}$$

To rotate a vector by an angle θ in two dimensions, it is multiplied by a rotation matrix given as follows

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \tag{7}$$

where $+Ve\ \theta$ corresponds to counterclockwise (CCW) and where $-Ve\ \theta$ corresponds to clockwise (CW) rotation.

Scenario 1: If **AB** is chosen as the tangent, then clockwise rotation should be applied. Then $\theta = -90°$. The corresponding rotation matrix is:

$$R(-90°) = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \tag{8}$$

Applying this rotation to the tangent vector gives the surface vector

$$\begin{aligned}\mathbf{S} &= R(-90^\circ)\,\mathbf{AB} \\ &= \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \end{bmatrix} \\ &= \begin{bmatrix} y_2 - y_1 \\ -(x_2 - x_1) \end{bmatrix}\end{aligned} \tag{9}$$

Scenario 2: If **BA** is chosen as the tangent, then counterclockwise rotation should be applied. Then $\theta = 90^\circ$.

$$\begin{aligned}\mathbf{S} &= R(90^\circ)\,\mathbf{BA} \\ &= \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \left(- \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \end{bmatrix} \right) \\ &= - \begin{bmatrix} -(y_2 - y_1) \\ x_2 - x_1 \end{bmatrix} \\ &= \begin{bmatrix} y_2 - y_1 \\ -(x_2 - x_1) \end{bmatrix}\end{aligned} \tag{10}$$

Therefore, both approaches produce the same surface vector expression. The difference lies only in the chosen orientation of the tangent vector and the corresponding rotation direction.

The **unit normal vector** is

$$\hat{\mathbf{n}}_e = \frac{\mathbf{S}_e}{\|\mathbf{S}_e\|} = \frac{\mathbf{S}_e}{\sqrt{S_{e,x}^2 + S_{e,y}^2}} \tag{11}$$

```
function [Sex, Sey, ...] = getSurfaceVectors(Nx, Ny, X, Y)
    % Initialize arrays
    Sex = zeros(Nx, Ny);
    Sey = zeros(Nx, Ny);
    nex = zeros(Nx, Ny);
    ney = zeros(Nx, Ny);
    % Loop over elements
    for i = 1:Nx
        for j = 1:Ny
            % East face - surface vector
            Sex(i,j) =   Y(i+1,j+1) - Y(i+1,j);
            Sey(i,j) = -(X(i+1,j+1) - X(i+1,j));
            % East face - unit normal surface vector
            nex(i,j) = Sex(i,j) / hypot(Sex(i,j), Sey(i,j));
            ney(i,j) = Sey(i,j) / hypot(Sex(i,j), Sey(i,j));
        end
    end
    % Same applies to other faces; North, South, West ...
end
```

Listing 3: Function to compute surface vectors

Note

For non-orthogonal grids, the surface vector $\mathbf{S}_f$ is decomposed as

$$\mathbf{S}_f = \mathbf{E}_f + \mathbf{T}_f,$$

where $\mathbf{E}_f \parallel \mathbf{CF}$ is the orthogonal component and $\mathbf{T}_f \perp \mathbf{CF}$ is the non-orthogonal correction component.

Face Centers

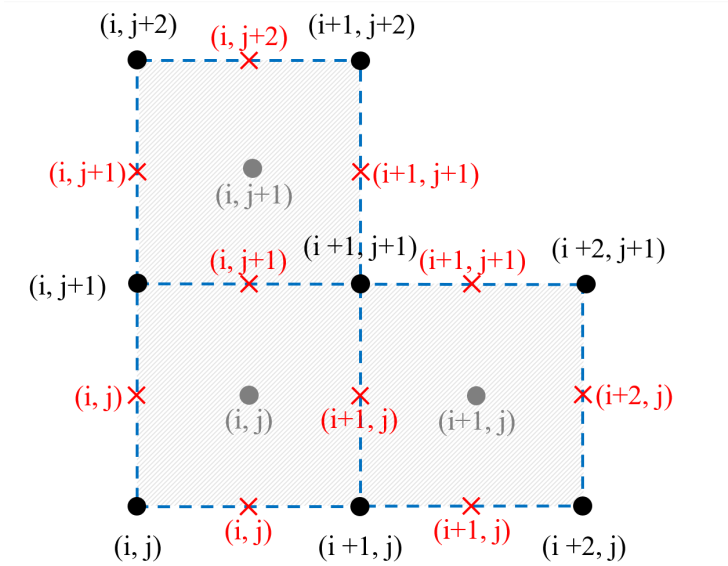

Figure 4: Domain

Face centers are the midpoints of each cell face. They are stored in separate arrays for *horizontal* faces (top/bottom of cells) and *vertical* faces (left/right of cells).

Horizontal Faces

$$\begin{aligned} X_{fH}(i,j) &= \frac{1}{2}\Big[X(i,j) + X(i{+}1,j)\Big], \\ Y_{fH}(i,j) &= \frac{1}{2}\Big[Y(i,j) + Y(i{+}1,j)\Big] \end{aligned} \tag{12}$$

Array size:$X_{fH},\ Y_{fH} \in \mathbb{R}^{N_x \times n_y}$

Vertical Faces

$$\begin{aligned} X_{fV}(i,j) &= \frac{1}{2}\Big[X(i,j) + X(i,j{+}1)\Big], \\ Y_{fV}(i,j) &= \frac{1}{2}\Big[Y(i,j) + Y(i,j{+}1)\Big] \end{aligned} \tag{13}$$

Array size: $X_{fV},\ Y_{fV} \in \mathbb{R}^{n_x \times N_y}$

Center-to-Center Distance Vector (CE)

The vector from the centroid of cell $C = (i,j)$ to the centroid of its east neighbor $E = (i{+}1,j)$ is

$$\mathbf{CE}(i,j) = \Big(X_C(i{+}1,j) - X_C(i,j),\ \ Y_C(i{+}1,j) - Y_C(i,j)\Big) \tag{14}$$

For interior cells, this connects two cell centroids. For **boundary cells** (e.g. $i = N_x$ on the east boundary), the neighbor centroid is replaced by the face center:

$$\mathbf{CE}(N_x, j) = \Big(X_{fV}(N_x{+}1, j) - X_C(N_x, j),\ \ Y_{fV}(N_x{+}1, j) - Y_C(N_x, j)\Big) \tag{15}$$

Note

An efficient implementation avoids `if`-statements inside the loop: run the loop from $i = 1$ to $N_x - 1$ for interior cells, then handle the boundary $i = N_x$ separately. Repeat analogously for all four directions (E, W, N, S).

Interpolation

Interpolation is required to estimate face values from cell-centered quantities in the finite volume formulation.

Geometric Interpolation

Uniform Grid

For two control volumes of equal size sharing a face, the face lies midway between the two cell centers. The face value reduces to the arithmetic mean:

$$\phi_f = 0.5\,\phi_C + 0.5\,\phi_E \tag{16}$$

Non-Uniform Grid

If the distances from the cell centroids to the shared face are unequal (say $2\Delta x$ on the left and Δx on the right):

$$\phi_f = \frac{(2\Delta x)\,\phi_C + (\Delta x)\,\phi_E}{2\Delta x + \Delta x} = \frac{2}{3}\,\phi_C + \frac{1}{3}\,\phi_E \tag{17}$$

General Interpolation Factor

Define a general interpolation factor g_f for each face as

$$g_f = \frac{Cf}{CF} \tag{18}$$

or

$$g_f = \frac{\cancel{V}_C}{\cancel{V}_C + V_F} \tag{19}$$

The face value can then be written compactly as

$$\boxed{\phi_f = g_f\,\phi_C + (1 - g_f)\,\phi_F} \tag{20}$$

Harmonic Interpolation

When discretizing the diffusion term, the diffusion coefficient must be evaluated at the face. For discontinuous or spatially varying diffusivity, the harmonic average is required to ensure flux continuity.

$$\frac{1}{\Gamma_f} = \left(\frac{g_f}{\Gamma_C} + \frac{1 - g_f}{\Gamma_F}\right) \tag{21}$$

Note

- If weighting factors will be computed based on distances, then the distance from the centroid to face centers for all elements should be calculated and stored - in addition to the center-to-center distance.
- the geometric factors (for all faces) should be calculated and stored, and then they can be used whenever interpolation is needed.
- For boundary faces, $g_f = 1$

```
function [ge, ...] = getInterpolationFactors(Nx,Ny,Vc)
    % Initialize arrays
    ge = zeros(Nx, Ny);
    % Loop over elements
    for i = 1:Nx
        for j = 1:Ny
            ge(i,j) = Vc(i,j)/(Vc(i,j)+Vc(i+1,j)) ;
            % This should be applied for all faces (e,w,s,n for quad
                elements) ...
        end
    end
end
```

Listing 4: Function to calculate interpolation factors

```
function [...] = interpolateToFace(Nx, Ny, phiC,interpType, ...)
    % Initialize arrays
    % phif is the value of phiC interpolated to the face f
    phie = zeros(Nx, Ny);
    phin = zeros(Nx, Ny);
    phiw = zeros(Nx, Ny);
    phis = zeros(Nx, Ny);
    % Loop over elements
    for i = 1:Nx
        for j = 1:Ny
            if strcmp(interpType, 'geometric')
                ...
            elseif strcmp(interpType, 'harmonic')
                ...
            end
        end
    end
end
```

Listing 5: Function to interpolate to faces

Gradients

The cell-centered gradient of a scalar field ϕ is computed using the Green–Gauss theorem:

$$\nabla\phi_C = \frac{1}{\forall_C}\sum_f \phi_f\,\mathbf{S}_f \tag{22}$$

where the sum runs over all faces f of the cell, ϕ_f is the interpolated face value, and $\mathbf{S}_f$ is the face surface vector.